

DBP Bahasa Melayu Advisory Chatbot

Beginner Edition - Existing Docker Host, No LM Studio

Deployment situation	Shared Ubuntu machine already running many Docker containers
Existing Docker observed	Docker Engine 29.5.1; Docker Compose v5.1.3
Host hardware	Intel Core Ultra 7 265K, 64 GB RAM, NVIDIA RTX 5090 32 GB
New architecture	Two isolated containers: Streamlit/RAG + llama.cpp/Gemma
Knowledge base	dbp_khidmatnasihat_clean_atomic - 33,320 items
Important change	Do NOT install LM Studio, Conda, Python or another Docker Engine on the host

WARNING: This replaces the previous llmster/LM Studio checklist. The faculty machine is shared and already runs other Docker applications, so the chatbot must be isolated as its own Docker Compose project.

STOP: Do not reinstall Docker. Do not edit /etc/docker/daemon.json. Do not restart Docker unless faculty IT explicitly approves it. Restarting Docker could affect their existing containers.

1. New Architecture

Everything specific to the thesis chatbot stays inside its own Docker Compose project:

```
Ubuntu shared host
|
+-- Existing faculty Docker applications  <-- DO NOT TOUCH
|
+-- dbp-chatbot Compose project
|
|   +-- chatbot container
|   |   +-- Streamlit
|   |   +-- Phase 4 retrieval/reranking
|   |   +-- Phase 5 generation client
|   |   +-- BGE-M3 + reranker (CPU initially)
|   |   +-- ChromaDB mounted read-only
|   |
|   +-- llm container
|       +-- llama.cpp CUDA server
|       +-- Gemma 3 4B IT GGUF
|       +-- RTX 5090
```

The chatbot calls llama.cpp over a private Docker network. The llama.cpp API port is not exposed to the faculty LAN. Only the Streamlit port is published.

NOTE: The existing Python code still uses variable names such as LMSTUDIO_BASE_URL. That name is retained only for compatibility; the URL now points to the llama.cpp container.

2. What You Should Bring / Use

- The populated deployment folder containing data/chroma_db/chroma.sqlite3 and all UUID folders.
- The updated shared-server deployment kit supplied for this architecture.
- The latest requirements.txt based on the working chatbotui environment.
- A few known-good thesis questions for final testing.

WARNING: Do not use the older deployment kit whose docker-compose.yml uses network_mode: host and expects llmster on 127.0.0.1:1234.

PHASE A - Read-Only Safety Check

These commands do not install, remove or restart anything. Paste the whole block:

```
echo "===== USER ====="
whoami

echo "===== DOCKER ====="
docker --version
docker compose version

echo "===== EXISTING CONTAINERS ====="
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"

echo "===== GPU ====="
nvidia-smi

echo "===== DOCKER GPU INFO ====="
docker info | grep -i -E "runtimes|nvidia|cdi" || true

echo "===== PORT 18501 CHECK ====="
ss -ltn | grep ":18501 " || true
```

CHECKPOINT A

- Docker version and Compose version should display normally.
- Existing faculty containers should remain running.
- nvidia-smi should show the RTX 5090.
- If the final port command prints nothing, port 18501 is probably free.

STOP: If nvidia-smi fails, STOP. If Docker commands fail, STOP. Do not reinstall Docker.

PHASE B - Test GPU Access in Docker Without Changing the Host

This runs a temporary container only. It does not install CUDA on the host. However, it may need to download an image if the image is not already cached.

```
docker run --rm --gpus all nvidia/cuda:12.8.0-base-ubuntu24.04 nvidia-smi
```

CHECKPOINT B

- The command should show the RTX 5090 from inside the temporary container.

STOP: If you get 'could not select device driver', 'unknown flag --gpus', or NVIDIA runtime errors, STOP and ask faculty IT whether NVIDIA Container Toolkit/GPU support is already configured. Do not modify their Docker daemon yourself.

PHASE C - Prepare the Chatbot Folder

Use Ubuntu Files (GUI) to copy the complete deployment folder from your USB/external SSD into your Home folder. For example:

```
/home/YOUR_USERNAME/DBP_Chatbot_Deploy_SharedServer
```

WARNING: YOUR_USERNAME is an example placeholder. Do not type it literally.

Open Terminal, then use:

```
cd ~/DBP_Chatbot_Deploy_SharedServer
pwd
ls -la
ls -lh data/chroma_db/chroma.sqlite3
```

CHECKPOINT C

- pwd should end with DBP_Chatbot_Deploy_SharedServer.
- docker-compose.yml, Dockerfile, requirements.txt and .env should be visible.
- data/chroma_db/chroma.sqlite3 must exist directly at that path.

STOP: If chroma.sqlite3 is under data/chroma_db/chroma_db/, fix the extra folder level before continuing.

PHASE D - Review the New Configuration Before Running Anything

```
cat .env
echo
cat docker-compose.yml
```

The important values should include:

```
LMSTUDIO_BASE_URL=http://llm:8080/v1/
LMSTUDIO_MODEL=google/gemma-3-4b
CHATBOT_HOST_PORT=18501
LLAMA_HF_REPO=lmstudio-community/gemma-3-4b-it-GGUF:Q4_K_M
```

The Compose file should contain two services named llm and chatbot. It should NOT contain network_mode: host.

NOTE: The llama.cpp model source above uses the LM Studio community Gemma 3 4B IT GGUF family with Q4_K_M. It is the same Gemma 3 4B instruction-tuned family, but the exact quantization used in the original thesis LM Studio installation should still be documented if later reproducibility needs to be exact.

PHASE E - Pull Only the llama.cpp Container Image

This does not touch existing faculty containers. It only downloads the image required by this Compose project.

```
docker compose -p dbp-chatbot pull llm
```

CHECKPOINT E

- The command should finish without an ERROR.

STOP: If registry access is blocked or the pull fails, copy the exact error. Do not change Docker settings.

PHASE F - Build Only the Chatbot Container

This builds your Python/Streamlit/RAG image from the Dockerfile.

```
docker compose -p dbp-chatbot build chatbot
```

The build can take time because it installs PyTorch CPU, ChromaDB, FlagEmbedding, Transformers and other Python dependencies.

CHECKPOINT F

- The build must finish successfully.

STOP: If it fails, run the same build again only after identifying the error. Do not run apt/pip installs directly on Ubuntu.

PHASE G - Start ONLY the LLM Container First

We start the model service separately so problems are easier to identify.

```
docker compose -p dbp-chatbot up -d llm
docker compose -p dbp-chatbot ps
docker compose -p dbp-chatbot logs -f --tail=100 llm
```

The first start may download the Gemma GGUF into this project's Docker volume. Wait for the model to finish downloading/loading.

NOTE: To exit the live log view without stopping the container, press Ctrl+C.

CHECKPOINT G

- The llm service should become healthy/running.
- Logs should indicate CUDA/GPU offload rather than CPU-only model execution.
- nvidia-smi should show a llama.cpp-related process/VRAM usage while the model is loaded.

STOP: If the model repeatedly restarts or does not detect CUDA, STOP and capture the logs. Do not restart the host Docker daemon.

PHASE H - Test llama.cpp from Inside the Docker Network

Because port 8080 is intentionally not published to the host, test it from a temporary container attached to the same Compose network:

```
docker run --rm --network dbp-chatbot_dbp-internal curlimages/curl:latest -fsS
http://llm:8080/v1/models
```

You should receive JSON describing the loaded model.

NOTE: If the network name differs, run 'docker network ls | grep dbp' and use the exact dbp-chatbot network name shown.

PHASE I - Verify ChromaDB

Run the verification script using the built chatbot image:

```
docker compose -p dbp-chatbot run --rm --no-deps chatbot python /app/verify_chroma.py
```

Expected result:

```
Collection      : dbp_khidmatnasihat_clean_atomic
Document count  : 33320
Expected count  : 33320
```

STOP: If the count is not exactly 33,320, STOP. Do not rebuild or re-embed the knowledge base.

PHASE J - Start the Full Chatbot Project

```
docker compose -p dbp-chatbot up -d
docker compose -p dbp-chatbot ps
docker compose -p dbp-chatbot logs --tail=150 chatbot
```

CHECKPOINT J

- llm should be running/healthy.
- chatbot should be running/healthy after startup.
- Existing faculty containers must remain unchanged.

PHASE K - Open the Chatbot Locally

On the Ubuntu machine, open a browser:

```
http://localhost:18501
```

Test known thesis questions, including one normal advice question and one refusal/out-of-domain question.

NOTE: For now, use port 18501 rather than 8501 to reduce the chance of clashing with an existing application.

PHASE L - Useful Commands for a Beginner

What you want to do	Command
See only our containers	<code>docker compose -p dbp-chatbot ps</code>
See chatbot logs	<code>docker compose -p dbp-chatbot logs --tail=200 chatbot</code>
See LLM logs	<code>docker compose -p dbp-chatbot logs --tail=200 llm</code>
Follow logs live	<code>docker compose -p dbp-chatbot logs -f</code>
Restart only our chatbot	<code>docker compose -p dbp-chatbot restart chatbot</code>
Stop/remove only our running project	<code>docker compose -p dbp-chatbot down</code>
Start our project again	<code>docker compose -p dbp-chatbot up -d</code>
See GPU use	<code>nvidia-smi</code>
See our resource use	<code>docker stats --no-stream</code>

STOP: Do NOT run 'docker system prune', 'docker container prune', 'docker volume prune', or commands that stop/remove all containers. This server belongs to other applications too.

WARNING: When stopping your project, do not add '-v' to docker compose down unless you intentionally want to delete the Gemma/model cache volume.

PHASE M - If Port 18501 Is Already Used

Check:

```
ss -ltn | grep ":18501 " || true
```

If something is using it, choose another unused high port, for example 18502 or 18510. Edit only this line in .env:

```
CHATBOT_HOST_PORT=18510
```

Then:

```
docker compose -p dbp-chatbot up -d
```

PHASE N - External Faculty URL Comes Later

Do not expose the chatbot publicly during the first local installation. First confirm localhost works and the known thesis answers behave correctly.

- Faculty IT can later map a fixed IP/domain to the Streamlit service.
- Use HTTPS and access control for external data-provider testing.
- Do not publish llama.cpp port 8080 to the Internet.
- Do not modify faculty firewall/reverse proxy without their administrator.

Troubleshooting Decision Table

Symptom	Meaning	Action
Docker commands work but GPU test fails	Host Docker lacks usable NVIDIA GPU integration	Stop and ask faculty IT; do not reconfigure daemon.
llm image pull fails	Registry/network restriction	Capture error; do not modify global Docker config.
llm downloads model but stays unhealthy	Model/CUDA/startup issue	Check 'docker compose -p dbp-chatbot logs --tail=200 llm'.
Chroma count is not 33,320	Wrong data folder/mount	Fix folder path; do not re-index.
Chatbot cannot call LLM	Internal Compose networking/model health issue	Check both service logs and /v1/models from dbp network.
localhost:18501 works	Core deployment succeeded	Only then discuss LAN/domain access.
Other faculty apps misbehave	Potential shared-host impact	Stop only our project with 'docker compose -p dbp-chatbot down' and inform IT.

Emergency: Remove Only Our Running Containers

From inside the DBP deployment folder:

```
docker compose -p dbp-chatbot down
```

This stops/removes the containers and network belonging to this Compose project. It does not intentionally stop the faculty's unrelated containers.

WARNING: Do not run global cleanup commands on a shared Docker server.

Official Technical Basis

Docker Compose is intended for defining and running multi-container applications. Docker supports GPU access for containers when the host daemon is configured for NVIDIA GPUs.

llama.cpp provides an OpenAI-compatible HTTP server and official CUDA server Docker images. The server supports model aliases, Hugging Face GGUF repositories, and GPU layer offload.

References: Docker Compose/GPU docs; llama.cpp server/Docker docs; LM Studio community Gemma 3 4B GGUF repository.

OK: Today's goal: reach a working local chatbot at <http://localhost:18501> without installing or changing host-level AI/Python software and without disturbing the faculty's existing Docker applications.